Conteo de Inversiones

Tengo un conjunto de n elementos y 2 arreglos/listas ordenados por diferentes criterios (`A` y `B`). Queremos dar una medida de semejanza entre ambos arreglos.

Dado que no nos importan los elementos en sí, sino su orden, simplemente vamos a nombrar a los elementos de `A` con $1, 2, 3, ...$, en orden, y los de B van a corresponder. Es decir, si `A = [elefante, gato, burro]` y `B = [burro, elefeante, gato]` entonces vamos a considerar que `A = [1, 2, 3]` y por lo tanto `B = [3, 1, 2]`.

Queremos ver qué tan diferente es el orden de `B` respecto al de `A`. Vamos a considerar como criterio contar la cantidad de inversiones. Una inversión es un par de elementos en un arreglo en el cual $b_i > b_j$, si $i < j$. Es decir, "b_i debería venir después de b_j, pero está viniendo antes, están invertidos". Notar que si la cantidad de inversiones es 0, implica necesariamente que el arreglo se encuentra ordenado. Si `B` no tiene inversiones, significa que tiene el exacto mismo orden que `A`, y entonces son el mismo arreglo.

Una forma trivial calcula la cantidad de inversiones en tiempo cuadrático (para cada elemento veo todos los siguientes).

Mejora y código

Una mejor solución es plantear un algoritmo similar a Mergesort: vamos ordenando, y cuando hacemos el "intercalar ordenado", cada vez que elegimos a un elemento de la derecha por sobre uno de la izquierda, es que necesariamente hay una inversión. En realidad, hay tantas inversiones por ese elemento de la derecha, por elementos que queden a la izquierda (cada uno de los elementos de la izquierda son necesariamente mayores a este elemento de la derecha).

Recomendamos revisar el ejemplo de la clase que se da en detalle, si no se termina de entender bien este paso.

```python
def contar_inversiones(arreglo): # solo necesitamos el arreglo B, ya de 1 a n
    _, inv = _contar_inversiones_rec(arreglo)
    return inv

def _contar_inversiones_rec(arreglo):
    if len(arreglo) < 2:
        return arreglo, 0
    izquierda = arreglo[len(arreglo)//2:]
    derecha = arreglo[:len(arreglo)//2]
    ordizq, inv_izq = _contar_inversiones_rec(izquierda)
    ordder, inv_der = _contar_inversiones_rec(derecha)
    ordenado, nuevas_inv = intercalar_ordenado(izquierda, derecha)
    return ordenado, nuevas_inv + inv_izq + inv_der

def intercalar_ordenado(izq, der):
    i = 0
    j = 0
    nuevo = []
    inversiones = 0
    while i < len(izq) and j < len(der):
        if izq[i] < der[j]:
            nuevo.append(izq[i])
            i += 1
        else:
            nuevo.append(der[j])
            j += 1
            inversiones += len(izq) - i
    # se acabo un arreglo
    inversiones += len(izq) - i
    nuevo.extend(izq[len(izq) - i:])
    nuevo.extend(der[len(der) - j:])
    return nuevo, inversiones
```

Complejidad

En sí nos va a quedar exactamente igual a MergeSort. El código es casi un mergesort pero que hace unas cuentitas (de tiempo constantes) extra:

$$T(n) = 2T\left(\frac{n}{2}\right) + \mathcal{O}(n) \rightarrow T(n) = \mathcal{O}\left(n \log n\right)$$